6CM504 Concurrency and communication

Understanding and modelling concurrency

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

This week

This week we will start exploring the basic building blocks of CSP processes. We will study what constitutes a CSP process, and how we can write one. The notation you will have read in the pre-seminar reading material generally uses the pure mathematical notation for CSP. In the seminars, we will use the machine-readable notation, known as CSP-M so that you may try out examples directly in the FDR tool that we will be using in the laboratory.

Table of Contents

Alphabets and communication

- A CSP process is defined by the way it communicates with its environment
- It communicates by engaging in events
  - The set of events in which a process may engage is known as its alphabet $\mathtt{A}$
  - The set Σ represents all the events in the world
  - Therefore for any given process. $\mathtt{A} \subseteq \Sigma$
- Engagement in an event is instantaneous
  - CSP is an untimed language
- An event occurs when both a process, and its environment, agree to engage in the event

Events

- Events are atomic and instantaneous
- An event models an important point in the history of a system
- If the duration of an action is important, then that action may be modelled by more than one event

Processes

- A process tells us when certain events may occur, and when they may be blocked
- It is a record of when the corresponding interactions are possible

Example

```
Alphabet = {lecture, eat, getup, sleep, eat}

Student =
  getup ->
    lecture ->
      laboratory ->
        eat ->
          sleep ->
            Student
```

STOP

- The most basic CSP process is the one which never does anything
- It is written: `STOP`
- It starts, never communicates anything, and never even terminates!
  - We will say more about the significance of *termination* when we study semantics later in the module

Table of Contents

Prefix

- Given an event a$\in \Sigma$, and a process P,

  ```
  a -> P
  ```

  is the process which is initially willing to communicate the event a, after which it behaves as the process P
- The arrow -> is the *prefix operator*—it prefixes an event to a process
- It will wait indefinitely for this a to happen before the communication occurs
- You may think of this as a button marked a, waiting to be pressed
- By convention, we use lower case letters to indicate events, and upper case letters for processes

Prefix: an example

- We can use STOP and prefixing to define processes that perform a fixed (finite) sequence of communications before stopping

```
Day =
  getup ->
    breakfast ->
      seminar ->
        lunch ->
          labwork ->
            dinner ->
              study ->
                bed -> STOP
```

- Question: What is the alphabet of the process Day?

Definitions

- We may define processes—or give names to processes—using equations
- We write:

```
A = B
```

to introduce a new process, named `A` which behaves exactly as the process `B`

Table of Contents

Recursion

- We may wish to write processes that, instead of quickly stopping, go on to do things indefinitely
  - In order to do this, we use *recursion*—the act of a process calling itself instead of terminating
- Consider the process

```
up -> down -> STOP
```

  We can give this process a name

```
P = up -> down -> STOP
```

  Instead of stopping, we can make it go on forever

```
P = up -> down -> P
```

Recursion

- Our process `P` has been defined recursively
- Any use of the recursively defined process name `P`

  ```
  P = up -> down -> P
  ```

  on the right hand side

  ```
  P = up -> down -> P
  ```

  means exactly the same as the whole
- The general form of a recursive definition is that an identifier is declared on the left hand side of the equation and the process term appears on the right

Mutual recursion

- We can use systems of equations that are *mutually recursive*—that is, they call each other
- For example

```
Pup = up -> Pdown

Pdown = down -> Pup
```

  behaves in exactly the same way as P—assuming we start with `Pup`
- In fact, as we will see later, CSP allows us to prove that systems of this nature are (or are not) exactly equivalent

Recursion: an example

- Car alarms are annoying
- When they go off, they start beeping and never seem to stop

```
CarAlarm = beep -> CarAlarm
```

State

- There is no implicit notion of state (in pure CSP)
  - In our CSP-M, we relax this requirement in order to code models in FDR
- Information about the state of a process maybe included in the left hand side of a definition

```
Crossing = Green
Green = request -> Request
Request = amber -> Amber
Amber = red -> Red
Red = flashing -> Flashing
Flashing = green -> Green
```

State

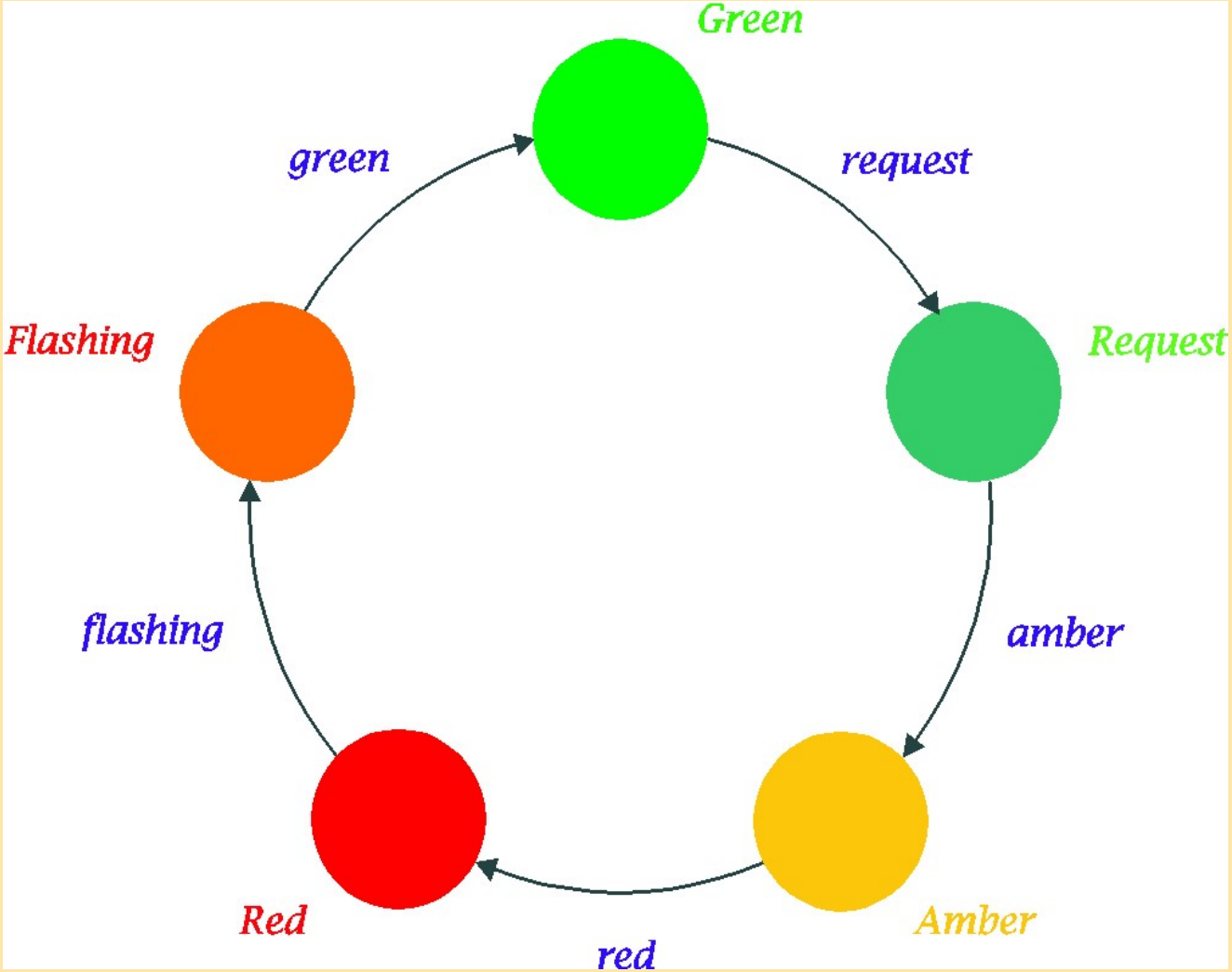
Green
green
request
Flashing
Request
flashing
amber
Red
Amber
red

Table of Contents

Guarded alternatives

- Until now, our processes have been rather straightforward—there has been only one event to engage with in any state
- In practice, we will want processes that are capable of offering a choice of events to their environments
- The choice operator `[]` takes a list of initial events prefixing processes and allows the environment to choose any one

```
( a1 -> P1 [] a2 -> P2 )
```

can do any either of `a1` or `a2` initially, and after behaves either as `P1` or `P2` respectively

More choices

- We are not limited to having two choices—the number of alternatives is arbitrary

```
( a1 -> P1 [] a2 -> P2 [] a3 -> P3 )
```

  offers any one of four alternatives
- The order of choices not significant

```
( a2 -> P2 [] a1 -> P1 [] a3 -> P3 )
```

  is the same process, with the same choices, and the same subsequent behaviours

Guarded alternatives

- The process

```
UpAndDown =
  up -> down -> STOP
  []
  down -> up -> STOP
```

  can engage in the events `up` and `down` in either order, after which it stops

Combining guarded alternatives and recursion

- Consider the mutually recursive processes, both of which contain guarded alternatives

```
P = a -> P [] b -> Q

Q = a -> P [] b -> STOP
```

- Question: can you (informally) describe the behaviour of these two mutually recursive processes?

Prefix choice

- *Prefix choice* is a generalised version of the guarded choice operator
- Assume $A \subseteq \Sigma$ is a set of events, and for each $x \in A$ we have defined a process `P(x)`, then

```
?x:A -> P(x)
```

  is the process which accepts any element `x` of `A` (any event `x` in the set of events `A`) and then behaves as the appropriate `P(x)`
- This generalises the guarded alternative construction to sets of events

Prefix choice

- We tend to use this generalised version when the dependency of `P(x)` upon `x` is through using `x` in constructing subsequent communications
- For example a process which repeats every communication might look like

```
Repeat = ?x:A -> x -> Repeat
```

- Or one which behaves similar to our counter, depending on the first communication might look like

```
Initialise = ?n:Nat -> Count(n)
```

Table of Contents

Finite vs infinite

- The difference between finite and infinite (and bounded/unbounded) is important: the tools we will use for investigating our processes are model-checkers that need to be able to visit every state

Finite vs infinite

- This example defines an infinite mutual recursion for every $n \in \mathbb{N}$

```
Count = Count(0)
Count(0) = up -> Count(1)

Count(n) = up -> Count(n+1)
           []
           down -> Count(n-1)
```

- This process will communicate any sequence of `up` and `down` as long as there has never been $n+1$ more `down` than `up`
- *This is an infinite state machine*
- The tools we are using will not, in general, be able to visit every state of an infinite state machine

Table of Contents

Summary

- In this lecture we have been introduced to the basic building blocks of CSP processes: events and processes
- We have seen how we can combine events using prefixing to create complex processes
- And how we can use (mutual) recursion to combine processes
- The choice operator gave us the possibility of having an option of multiple events in any one state
- In the next lecture we shall look at the choice operator in more detail, and study different types of choice, their implications, and some laws that govern choice

Summary

- Advance reading: Schneider, pages 11–28
- Advance reading: Roscoe, pages 13–28
- Advance reading: Woodcock and Davies, chapter 2